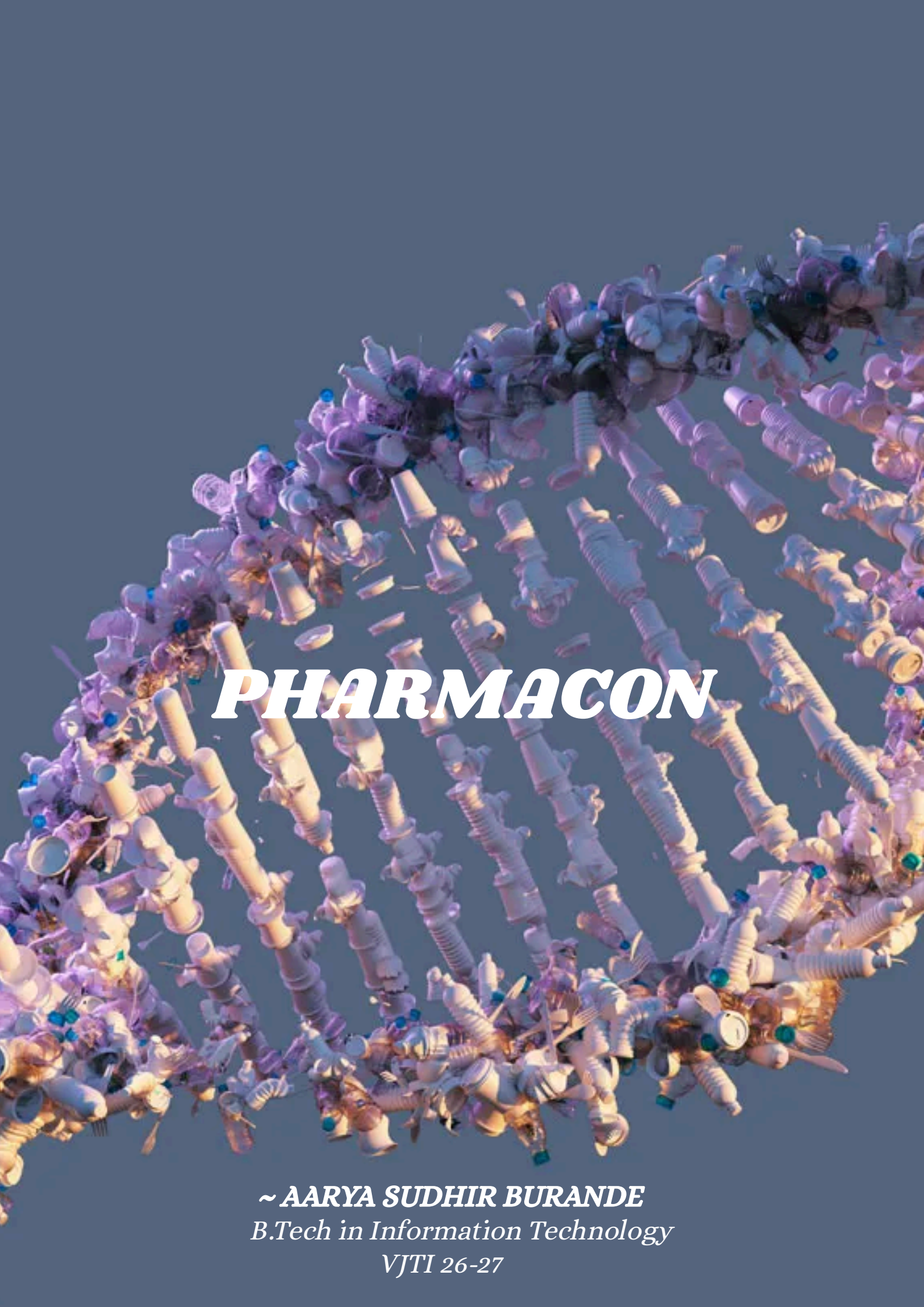




PHARMACON

~ AARYA SUDHIR BURANDE
B.Tech in Information Technology
VJTI 26-27

ABSTRACT:

The Greek word *pharmacon* means, simultaneously, medicine and poison. A drug meant for healing, when administered with an incompatible drug, can act as a poison. With over 14,000 approved drugs on the market, the possible drug pairs run into the tens of millions—far beyond what clinical trials can cover.

The challenge lies in representing drug molecules in a way that captures both their local atomic geometry and broader chemical identity. Current approaches are divided: graph methods capture local structure but miss global context, while sequence-based methods recover that context but lose the geometry that governs molecular interaction. The gap between them is where the most chemically meaningful information lives.

This project combines two complementary machine learning approaches for predicting drug-drug interactions. A transformer based model learns chemical information directly from molecular SMILES representations, capturing important atoms, substructures, and functional groups within a drug. At the same time, a graph neural network models biological relationships between drugs, proteins, and protein interaction networks to understand how drugs behave within the body. These two representations are fused using an attention-based mechanism that allows the model to determine whether a predicted side effect is driven more strongly by chemical structure or biological interactions. By preserving information at multiple levels on both the chemical and biological sides, the system aims not only to predict adverse drug interactions but also to explain the molecular and biological factors responsible for them.

ABOUT ME :

I am Aarya S. Burande, and I recently completed my first year of B.Tech in IT Engineering at VJTI. My interests lean towards AI architectures and modelling, I really enjoy figuring out how different models work. The satisfaction of finally unravelling a puzzle and optimizing it to solve a real-world problem is immensely rewarding. Moreover, the chance to actually design those puzzles is unparalleled.

Currently, I mostly work with C++ and Python, and I enjoy exploring different fields whenever I get the chance. While researching for this project, I stumbled across several topics that I found incredibly fascinating. I often had to stop myself from disappearing down rabbit holes just to stay focused on the project itself. That said, those rabbit holes are exactly what made the experience exciting, and I would love to learn more about those areas in the future.

I picked this project specifically because it brought together several topics that I was curious about, such as AI architectures, graph models, and drug interactions. The more I read about it, the more questions I had. To be honest, it seemed really cool, and I wanted to be a part of the group working on it. I was very interested in DNA interactions and protein structures back in 10th grade, so in a way, this project reconnected me with an older curiosity while introducing me to entirely new ideas.

OBJECTIVE :

The goal of this project is to explore whether two different representations of a drug—its chemical structure and its biological interactions—can be combined into a single framework to better understand and predict drug-drug interactions. Existing approaches often focus on one representation or the other, so we aim to investigate whether bringing them together can provide a more complete picture of how drugs interact within the body.

In addition to prediction, a major focus of the project is interpretability. Rather than treating the model as a black box, we aim to build an interpretability layer that can trace a predicted side effect back to the factors that may have contributed to it. These factors could be chemical, such as specific atoms, substructures, or functional groups, or biological, such as proteins, pathways, and interaction networks. The objective is not only to predict that an interaction may occur, but also to provide insight into why it may occur.

PROJECT FLOW:

Data Collection

- Drug SMILES
- Drug-Protein Interactions
- Protein-Protein Interactions
- Protein Sequences
- KEGG / Reactome / GO Data



Data Preprocessing

- RDKit Processing
- Functional Group Extraction
- Substructure Mapping
- Graph Construction
- Data Cleaning



Chemical Representation Learning

- Custom Molecular Transformer
- Token Embeddings
- Substructure Embeddings
- Functional Group Embeddings
- Molecular Embeddings



Biological Representation Learning

- ESM Protein Embeddings
- Drug-Protein Graph
- Protein-Protein Graph
- Pathway-Level Representation
- GNN Training

Representation Fusion

- Preserve Chemical Features
- Preserve Biological Features
- Attention/Gated Fusion
- Contribution Analysis



Drug-Drug Interaction Prediction

- Multi-Relational Prediction
- Side Effect Classification
- Confidence Analysis



Interpretability Framework

- Atom-Level Attribution
- Substructure Attribution
- Functional Group Attribution
- Protein Attribution
- Pathway Attribution



Evaluation & Validation

- Baseline Comparisons
- Ablation Studies
- Cold-Start Evaluation
- Perturbation Testing
- Performance Analysis



DATASETS :

The proposed framework requires data from multiple sources to capture both the chemical and biological aspects of drug-drug interactions.

1. Drug SMILES Data

- Source: DrugBank
- Purpose: To obtain the molecular structure of drugs in SMILES format for transformer-based molecular representation learning.

2. Drug-Drug Interaction (DDI) Data

- Source: TWOSIDES / DrugBank
- Purpose: To obtain known drug-drug interactions and associated side effects for model training and evaluation.

3. Drug-Protein Interaction (DPI) Data

- Source: DrugBank / STITCH
- Purpose: To identify the protein targets of drugs and construct the drug-protein interaction network.

4. Protein-Protein Interaction (PPI) Data

- Source: STRING Database
- Purpose: To model biological relationships between proteins and build the protein interaction network.

5. Protein Sequence Data

- Source: UniProt
- Purpose: To generate protein embeddings using the ESM protein language model.

6. Pathway Data

- Source: KEGG and Reactome
- Purpose: To map proteins to biological pathways and construct pathway-level representations.

7. Biological Process Data

- Source: Gene Ontology (GO)
- Purpose: To associate proteins and pathways with higher-level biological processes for interpretability.

THEORY AND APPROACH :

1. Molecular Representation Learning

DRUG REPRESENTATION

```
graph TD; A([DRUG REPRESENTATION]) --> B[Molecular (SMILES-based) Representation:]; A --> C[Biological (Graph-based) Representation:]; B --> D[Why there is a need to combine both representations?]; C --> D;
```

Molecular (SMILES-based) Representation:

The SMILES (Simplified Molecular Input Line Entry System) representation describes a molecule using a sequence of characters. This allows transformer models to learn directly from the drug's structure and identify important chemical features such as atoms, substructures, and functional groups.

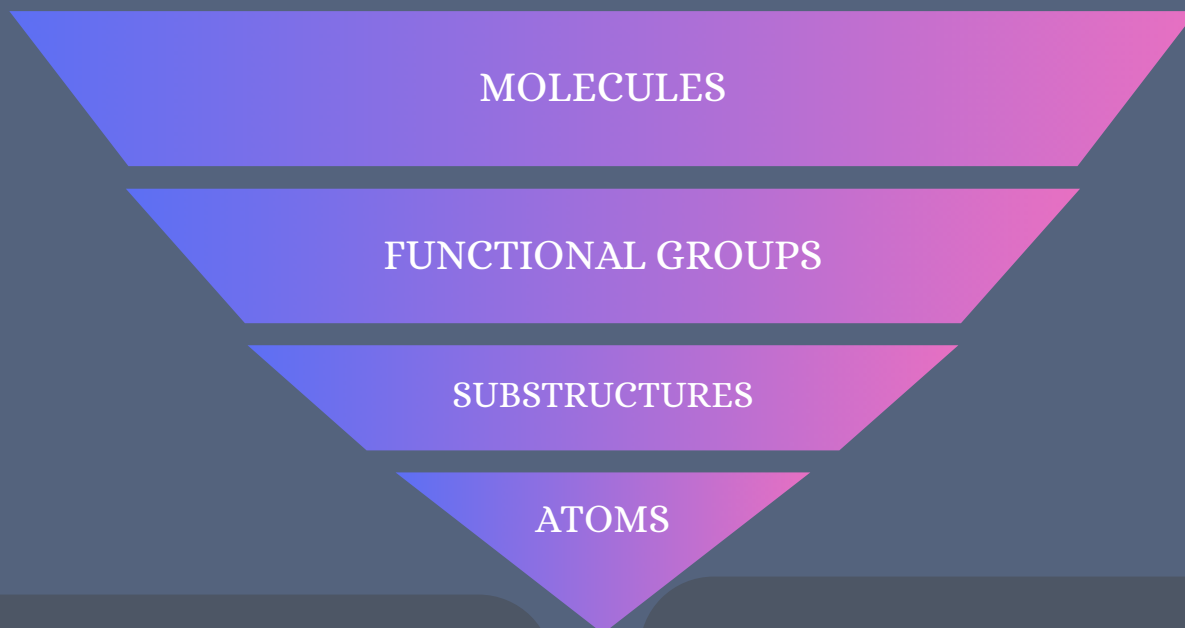
Biological (Graph-based) Representation:

Another way to represent a drug is through its biological interactions. Drugs interact with proteins, proteins interact with other proteins, and together they form complex biological networks. Graph Neural Networks can learn from these relationships and capture how a drug behaves inside the body.

Why there is a need to combine both representations?

Both representations provide different information. SMILES tells us what the drug is chemically, while the graph tells us how the drug interacts biologically. Since drug interactions depend on both chemical structure and biological context, combining these representations may provide a more complete understanding of drug-drug interactions than either approach alone.

ESTABLISHING CHEMICAL HEIRARCHY:



ATOM/TOKEN:

The transformer reads the SMILES string and generates embeddings for individual tokens

Example:

CC(=O)O

contains carbon and oxygen atoms represented through tokens.



SUBSTRUCTURES:

Using RDKit, we identify meaningful molecular fragments such as:

Benzene Ring

Carbonyl Group

Alkyl Chain

and aggregate the corresponding token embeddings.



MOLECULES:

Finally, all the information is combined into a single molecular embedding representing the entire drug.



FUNCTIONAL GROUPS:

Substructures are further grouped into chemically meaningful functional groups such as:

Alcohol

Amine

Amide

Carboxylic Acid

Sulfonamide

These are often the parts of a molecule responsible for specific chemical behavior.

CHEMICAL HEIRARCHCY AND INTERPRETABILITY:

WHY DO WE NEED A CHEMICAL HEIRARCHY?

A major goal of the project is interpretability. The reason we use this hierarchy is that it lets us explain predictions at multiple levels.

Instead of:

Dimension 247 of the embedding caused the side effect.

we can say:

The model identified the sulfonamide functional group and aromatic ring structure as important contributors to the prediction.

That's the whole interpretability story on the chemistry side.

WHAT IS RDKit AND WHY WE USE IT HERE?

RDKit is a well-established, industry-standard cheminformatics toolkit. Written in C++ with extensive Python APIs, it is widely used for molecular property calculation, similarity searching, machine learning, and structural analysis.

RDKit helps us understand the chemistry of a molecule.

The transformer can learn patterns from a SMILES string, but it doesn't automatically know:

This part is a benzene ring.

This part is an amide group.

RDKit does.

So in our project:

SMILES

↓

RDKit

↓

Identify atoms, substructures,
and functional groups

Then we use those identified structures to organize the transformer's embeddings into our chemical hierarchy.

BIOLOGICAL REPRESENTATION LEARNING:

WHY DO WE NEED BIOLOGICAL REPRESENTATION?

A drug's chemical structure tells us what it is made of, while its biological interactions tell us how it behaves inside the body. Since drug-drug interactions are often caused by shared proteins, pathways, and biological processes, incorporating biological representations helps the model understand mechanisms that cannot be inferred from structure alone.

HOW DO WE REPRESENT THESE BIOLOGICAL ASPECTS LIKE DDI, DPI AND PPI?

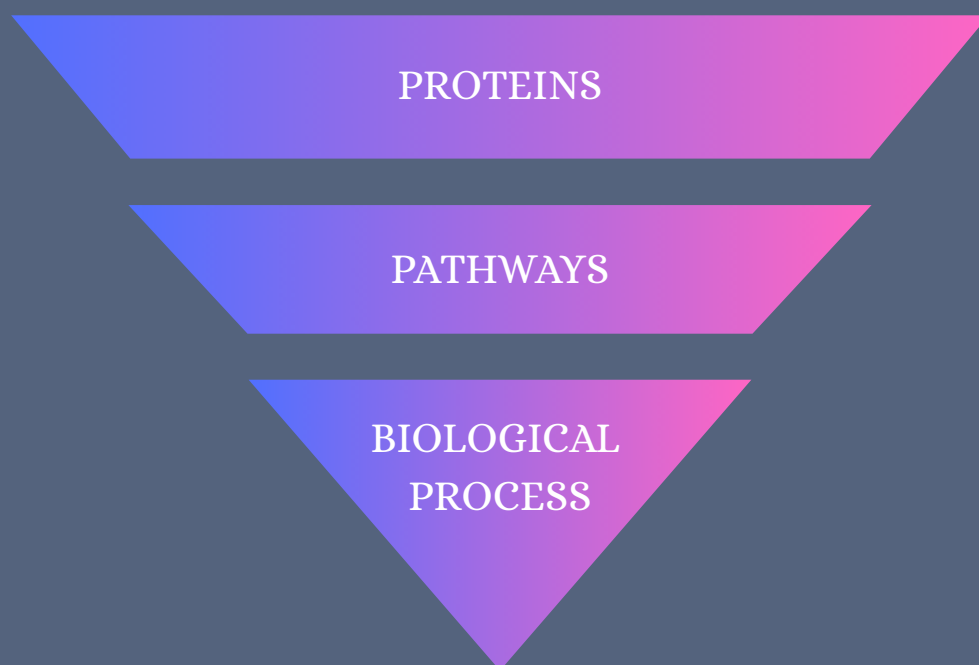
To capture the biological side of drug interactions, we represent drugs, proteins, and pathways as nodes in a graph. The connections between them represent known biological relationships. A drug may target a particular protein (Drug-Protein Interaction), proteins may interact with one another (Protein-Protein Interaction), and certain drug pairs may be associated with specific side effects (Drug-Drug Interactions). By organizing this information as a graph, we can use a Graph Neural Network to learn how information flows through these relationships. This helps the model understand not just what a drug looks like chemically, but also how it behaves within a complex biological system.

EMBEDDINGS: DRUGS AND PROTEINS

REPRESENTATION IN A GNN:

To initialize the graph, each drug is first represented using the embedding generated by the molecular transformer from its SMILES representation. These embeddings capture the drug's chemical properties and serve as the initial features of the drug nodes. Similarly, proteins are represented using embeddings generated by a protein language model such as ESM, which captures information about their structure and function directly from their amino acid sequences. Once these embeddings are assigned to the drug and protein nodes, a Graph Neural Network propagates information through the interaction network, allowing drugs to learn from connected proteins and pathways, and proteins to learn from their biological context. This produces graph-updated embeddings that capture the biological behaviour of each drug. Rather than replacing the original molecular embeddings, both representations are preserved and combined using an attention-based fusion mechanism. This allows the model to dynamically determine how much importance should be given to chemical structure versus biological context for a particular prediction, creating a more complete representation for downstream drug-drug interaction prediction.

BIOLOGICAL HEIRARCHY:



Protein Level

The foundation of the biological hierarchy is the protein level, since drugs exert their effects by interacting with proteins. To represent proteins, we use embeddings generated by ESM (Evolutionary Scale Modeling), a protein language model trained on large collections of protein sequences. Similar to how a transformer learns patterns from natural language, ESM learns structural and functional information from amino acid sequences. This allows each protein to be represented as a numerical embedding that captures its biological characteristics, providing richer information than simple identifiers or manually engineered features.



Pathway Level

Proteins rarely function independently and are usually part of larger biological pathways. To capture these relationships, proteins are grouped into pathways using curated biological databases such as KEGG and Reactome. These databases contain experimentally validated information about which proteins participate in specific cellular processes. By aggregating information from proteins belonging to the same pathway, the model can learn pathway-level representations that describe broader biological mechanisms rather than individual protein interactions.

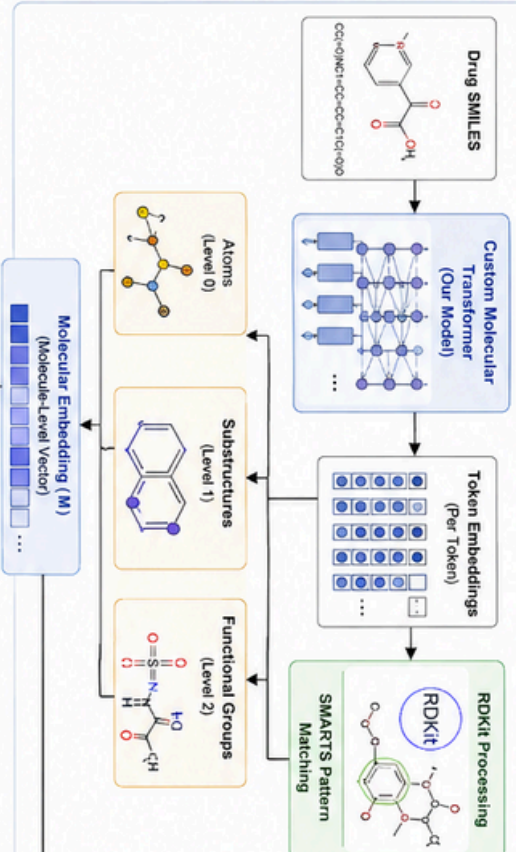


Biological Process Level

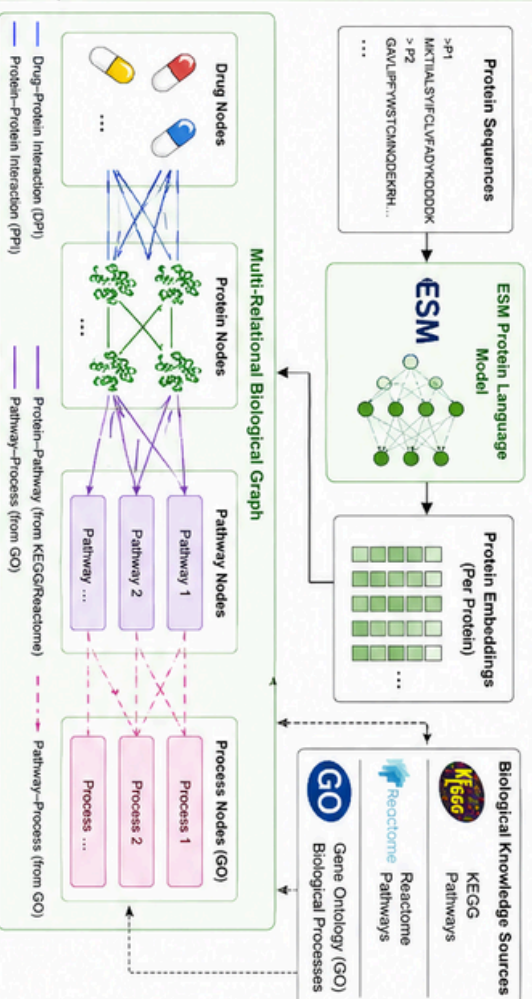
At the highest level, pathways contribute to larger biological functions such as inflammation, immune response, cell signaling, blood coagulation, or drug metabolism. To represent these processes, we use Gene Ontology (GO), which organizes biological knowledge into hierarchical categories of biological processes. Mapping proteins and pathways to GO terms allows the model to connect individual interactions to larger physiological functions. This provides a more interpretable explanation of predictions by showing not only which proteins were involved, but also which biological processes may have contributed to a particular side effect.

ARCHITECTURE:

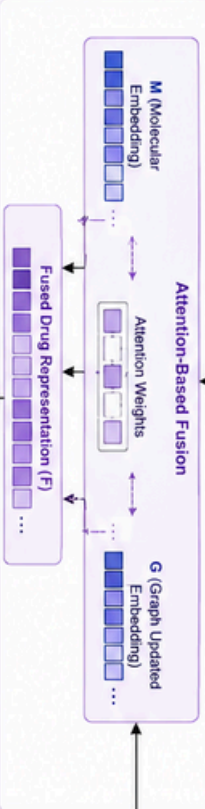
1. CHEMICAL BRANCH (Molecular Representation)



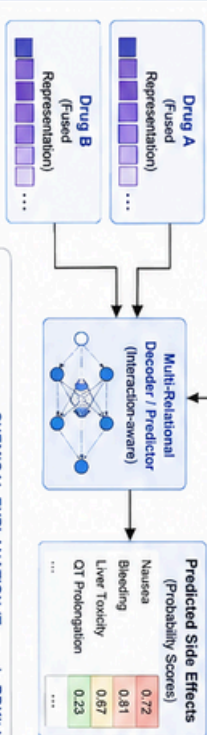
2. BIOLOGICAL BRANCH (Graph Construction & Biological Representation)



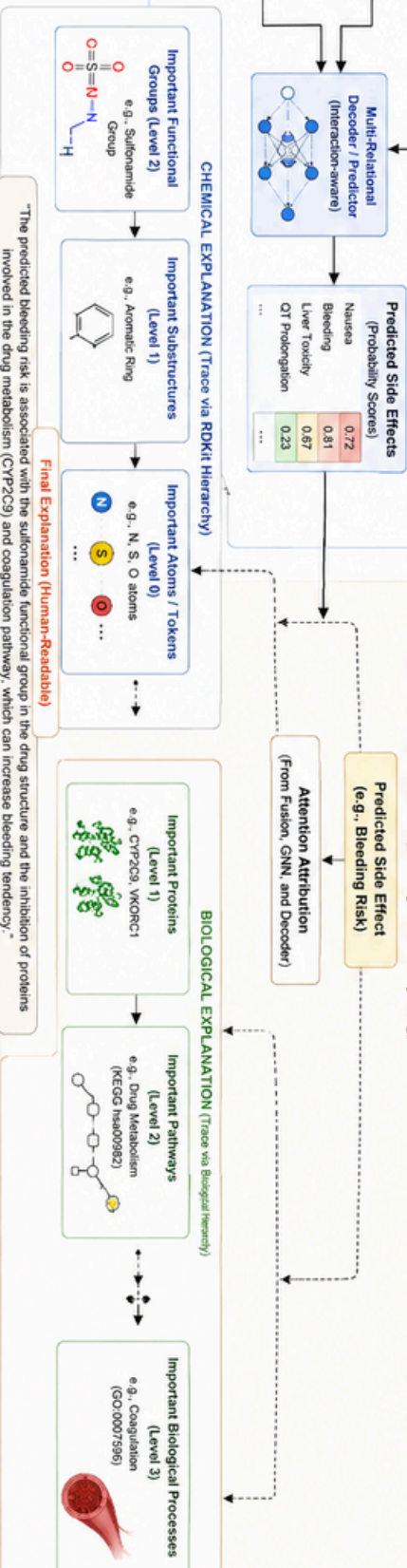
3. FUSION: CHEMISTRY + BIOLOGY



4. PREDICTION: DRUG-DRUG INTERACTION



5. INTERPRETABILITY LAYER (Tracing Back Everything)



WORKFLOW:

→ Week 1

- Literature review on Drug-Drug Interaction (DDI) prediction
- Study Decagon architecture and multi-relational graphs
- Study MOLFORMER and molecular transformers
- Identify limitations of existing approaches
- Finalize project objectives and scope

→ Week 2

- Study protein language models (ESM)
- Study Graph Neural Networks and message passing
- Explore KEGG, Reactome, and Gene Ontology databases
- Design preliminary architecture

→ Week 3

- Collect DDI, DPI, and PPI datasets
- Collect protein sequence data
- Collect pathway and biological process data
- Organize and clean datasets

→ Week 4

- Process molecular data using RDKit
- Generate atom, substructure, and functional group mappings
- Construct biological graph structure
- Define node and edge types

→ Week 5

- Develop custom molecular transformer
- Generate token-level embeddings
- Generate molecular embeddings
- Compare design choices with MOLFORMER

→ Week 6

- Build chemical hierarchy
 - Atom level
 - Substructure level
 - Functional group level
 - Molecule level
- Validate chemical representation pipeline

► Week 7

- Generate protein embeddings using ESM
- Initialize protein nodes
- Initialize drug nodes using transformer embeddings
- Integrate pathway information

► Week 8

- Implement Decagon-inspired multi-relational GNN
- Model DPI and PPI relationships
- Perform graph representation learning

► Week 9

- Generate graph-updated embeddings
- Design attention-based fusion mechanism
- Fuse molecular and biological representations

► Week 10

- Implement multi-relational side-effect prediction module
- Train prediction framework
- Optimize model architecture

► Week 11

- Build interpretability layer
- Trace predictions to:
 - Atoms
 - Substructures
 - Functional groups
 - Proteins
 - Pathways
 - Biological processes

► Week 12

- Baseline comparison with Decagon and MOLFORMER
- Ablation studies
- Perturbation testing for explanations
- Analyze results
- Documentation and final report

PROJECT RELATED TASK:

As a preparatory task for the proposed Drug-Drug Interaction (DDI) prediction project, the core Transformer architecture from the paper Attention Is All You Need was implemented from scratch using PyTorch.

The implementation included Multi-Head Attention, Positional Encoding, Feed Forward Networks, Encoder and Decoder layers, and the complete Transformer model. A simplified version of the architecture was trained on an English-German translation dataset to verify that the model was functioning correctly. The decreasing training loss across epochs confirmed successful learning.

This exercise helped build a practical understanding of the Transformer architecture, which serves as the foundation for many modern deep learning models, including those used in biomedical applications such as DDI prediction.

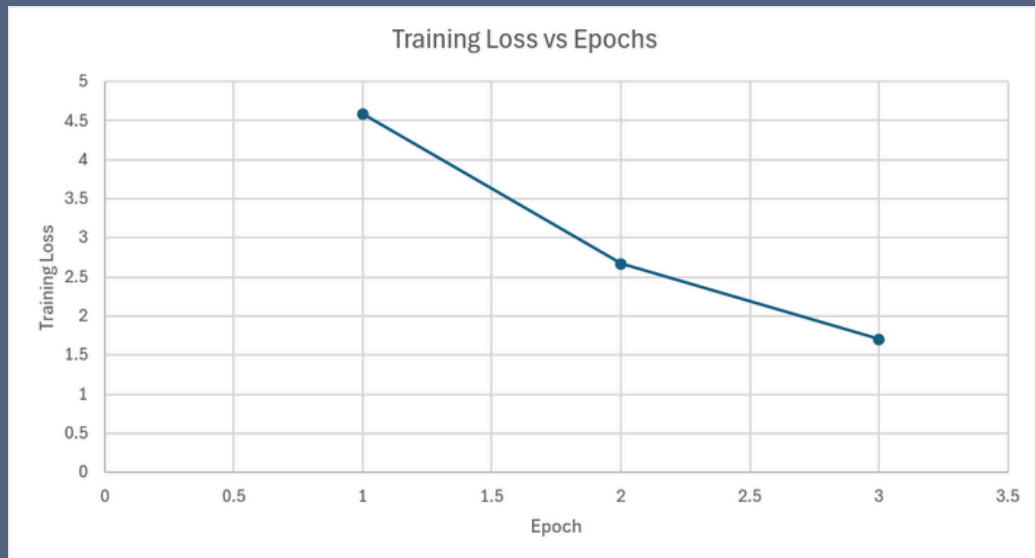
Source code architecture

```
.
├── README.md
├── PAPER_NOTES.md
├── requirements.txt
├── src
│   ├── attention.py
│   ├── multihead_attention.py
│   ├── positional_encoding.py
│   ├── feedforward.py
│   ├── encoder.py
│   ├── decoder.py
│   ├── transformer.py
│   ├── dataset.py
│   └── train.py
└── results
    ├── dataset.png
    ├── model_architecture.png
    ├── training_output.png
    └── training_loss_curve.png
```

Training output

```
(.venv) PS C:\Users\Aarya\Desktop\task3\src> py .\train.py
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Epoch 1/3 | Loss: 4.6148
Epoch 2/3 | Loss: 2.6708
Epoch 3/3 | Loss: 1.6753
Model saved successfully!
(.venv) PS C:\Users\Aarya\Desktop\task3\src>
```

Training Loss vs Epochs graph



*Note: Due to file size constraints, the complete project has been uploaded to a GitHub repository. The repository contains the complete source code, documentation, implementation details, and result screenshots for further review.

GitHub Repository: https://github.com/aaryasb3101/Project-X_task-3

REFERENCES:

Research Papers:

- [1] T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," International Conference on Learning Representations (ICLR), 2017. Available: <https://arxiv.org/abs/1609.02907>
- [2] A. Vaswani, N. Shazeer, N. Parmar, et al., "Attention Is All You Need," Advances in Neural Information Processing Systems (NeurIPS), 2017. Available: <https://arxiv.org/abs/1706.03762>
- [3] M. Zitnik, M. Agrawal, and J. Leskovec, "Modeling Polypharmacy Side Effects with Graph Convolutional Networks," Bioinformatics, vol. 34, no. 13, pp. i457–i466, 2018. Available: <https://arxiv.org/abs/1802.00543>
- [4] J. Ross, B. Belgodere, V. Chenthamarakshan, I. Padhi, Y. Mroueh, and P. Das, "Large-Scale Chemical Language Representations Capture Molecular Structure and Properties," Nature Machine Intelligence, 2022. Available: <https://arxiv.org/abs/2106.09553>

Databases & Datasets:

- [7] DrugBank Online Database. Available: <https://go.drugbank.com/>
- [8] TWOSIDES Drug-Drug Interaction Dataset.
- [9] SIDER: Side Effect Resource. Available: <http://sideeffects.embl.de/>
- [10] STRING: Functional Protein Association Networks. Available: <https://string-db.org/>
- [11] UniProt: Universal Protein Resource. Available: <https://www.uniprot.org/>
- [12] KEGG: Kyoto Encyclopedia of Genes and Genomes. Available: <https://www.genome.jp/kegg/>
- [13] Reactome Pathway Database. Available: <https://reactome.org/>
- [14] Gene Ontology Resource. Available: <https://geneontology.org/>

Software & Resources:

- [15] RDKit: Open-Source Cheminformatics Software. Available: <https://www.rdkit.org/>
- [16] PyTorch. Available: <https://pytorch.org/>
- [17] PyTorch Geometric (PyG). Available: <https://pytorch-geometric.readthedocs.io/>
- [18] ESM (Evolutionary Scale Modeling). Available: <https://github.com/facebookresearch/esm>
-